UNIVERSIDAD TECNOLOGICA NACIONAL | FACULTAD REGIONAL SAN FRANCISCO

Tecnicatura Universitaria en Programación

Programación 2

Plan 2024

Eje temático Nº 2

Índice

Índice.....2
Base de datos.....3
SQL Server Management Studio (SSMS).....7
Creación de base de datos.....7
Creación de tablas:.....9
Establecer clave primaria:.....9
Campo autonumérico:.....11
Conectar la base de datos con Visual Studio.....14
Cadena de conexión:.....16
Introducción a Servicios Web.....17
REST.....17
REST - Características.....18
API.....18
APIs No Web.....18
APIs Web (Arquitectura REST).....20
API REST.....20
API RESTful.....20
API REST vs API RESTful.....21
En resumen:.....21
Servidor.....21
Hosting.....22
Cuentas de dominio.....22
Certificado SSL.....23
Protocolos de conexión.....24
Git (gonza).....25
¿Para qué se usa?.....25
Repositorios.....26
Repositorios locales y remotos.....26
Commit.....26
Instalación.....27
GitHub Web y Desktop.....28
GitHub Desktop.....29
Comandos Básicos de Git.....29

Unidad Nº 2: Acceso a datos

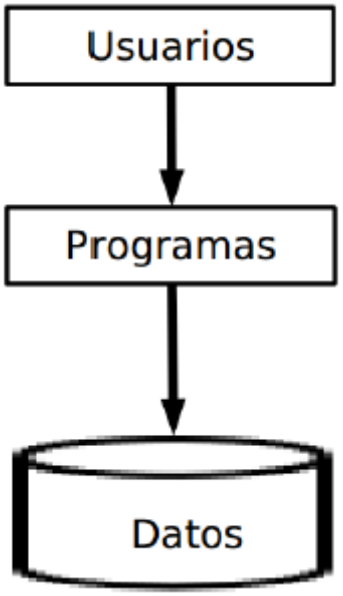
Base de datos

¿Qué es? Es una colección de datos relacionados, y una descripción de estos datos, diseñados para cumplir con las necesidades de información de una organización. (Connolly & Begg)

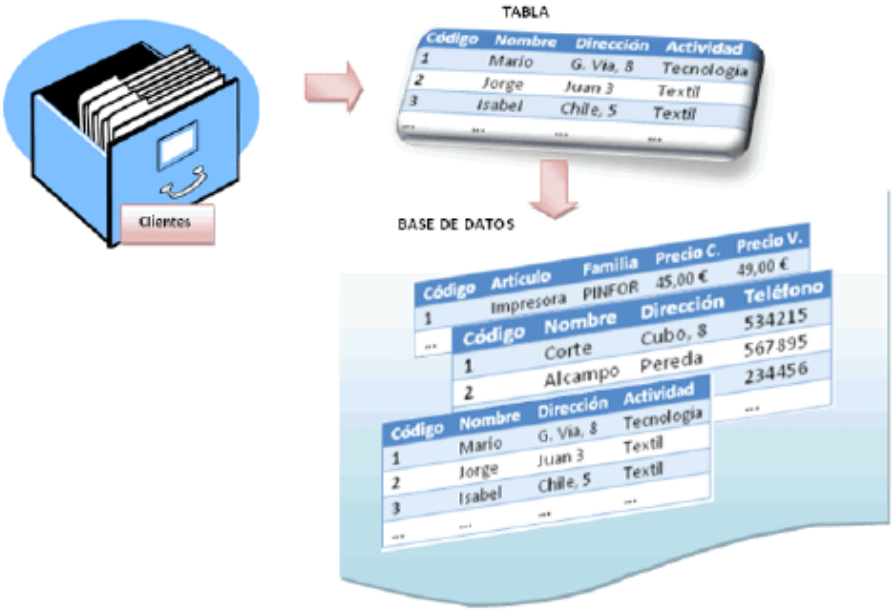
Para facilitar la administración de las bases de datos, podemos utilizar los **Sistemas Gestores de Bases de Datos** (DBMS por sus siglas en inglés). Lo definimos como un sistema de software que permite a sus usuarios crear, mantener y controlar el acceso a una base de datos. (Connolly & Begg)

El acceso a datos permite a los usuarios obtener datos guardados en una base de datos mediante la utilización de programas que vamos a desarrollar para cumplir con las necesidades del usuario.

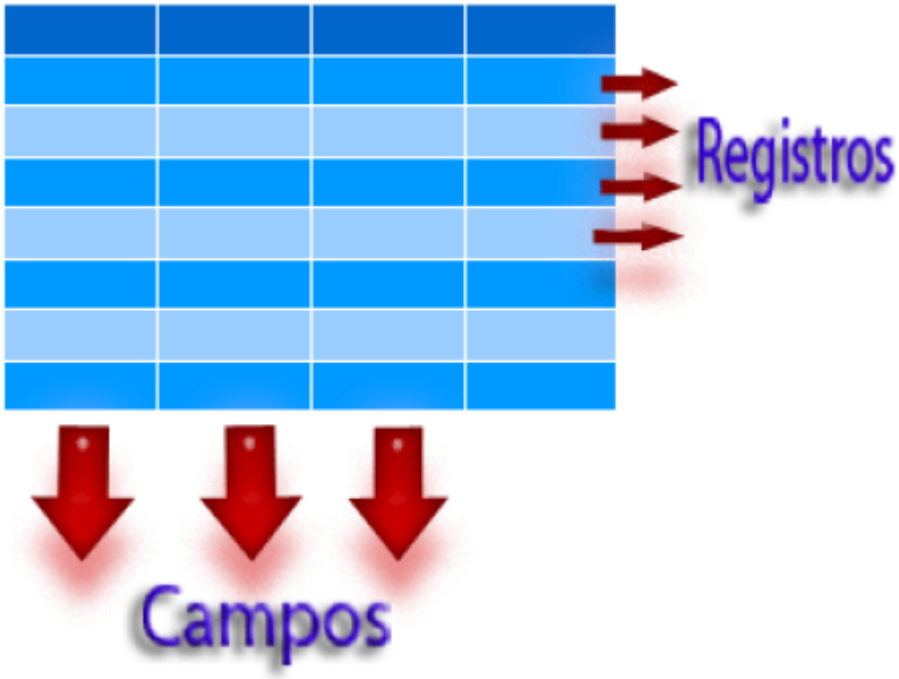
Flujo de acceso a los datos



Las bases de datos están formadas por tablas que almacenan los datos de las entidades.



Cada tabla se estructura de la siguiente manera:



Cada registro está constituido por diversos campos y cada campo, contiene el mismo tipo de información en cada uno de los registros, solo que los datos albergados en cada campo son distintos para cada registro

Para poder acceder a bases de datos SQL Server desde C#, debemos hacer uso de los siguientes espacios de nombre:

- System.Data;
- System.Data.SqlClient;

Para establecer la conexión a una base de datos SQL Server se utiliza la clase **SqlConnection**, a través de la cual podemos abrir la conexión con el servidor de bases de datos deseado.

Para poder indicar a nuestra conexión cuál es la base de datos que va a utilizar, debemos setear la cadena de conexión que incluye datos como: servidor al que se va a conectar, nombre de la base de datos, tipo de credenciales de acceso a utilizar (Usuario SQL Server o Usuario de Windows)
Ejemplo de cadena de conexión utilizando autenticación de windows:
Data Source=miEquipo\sqlexpress;Initial Catalog=MiBaseDeDatos;Integrated Security=True;
Una vez creada la cadena de conexión debemos asignarla a la propiedad ConnectionString de nuestro objeto SqlConnection de la siguiente manera:
conn.ConnectionString = @"**Data Source=miEquipo\sqlexpress;Initial Catalog=MiBaseDeDatos;Integrated Security=True;**";

La clase **SqlDataAdapter** recibe como parámetros un conjunto de comandos de datos y una conexión de base de datos que se utilizan para rellenar un DataTable y actualizar una base de datos de SQL Server. Esta clase no se puede heredar.

La creación de éste objeto se realiza de la siguiente manera:
SqlDataAdapter da = new SqlDataAdapter(query, conn);
Dónde query es un string con la sentencia sql y conn es un objeto SqlConnection con la conexión ya abierta.
Una vez ejecutada la consulta, los datos obtenidos como resultado deben ser guardados en un DataTable para que puedan ser utilizados desde C#. Dicha operación se realiza mediante el método **Fill** de nuestro objeto SqlDataAdapter como se muestra a continuación:
da.Fill(dt);
dt es un objeto de tipo DataTable el cual representa una tabla de datos en memoria

Pasos para realizar la ejecución de una consulta:

- Abrir Conexión
- Hacer consulta
- Relacionar consulta con conexión (SqlDataAdapter)
- Crear DataTable
- Agrega o actualiza filas en DataTable (SqlDataAdapter.Fill(DataTable))
- Cerrar Conexión

SQL Server Management Studio (SSMS)

Es una herramienta de administración integrada desarrollada por Microsoft para gestionar bases de datos en SQL Server. Ofrece una interfaz gráfica de usuario (GUI) que permite a los administradores de bases de datos, desarrolladores y otros usuarios realizar una amplia variedad de tareas en servidores SQL.

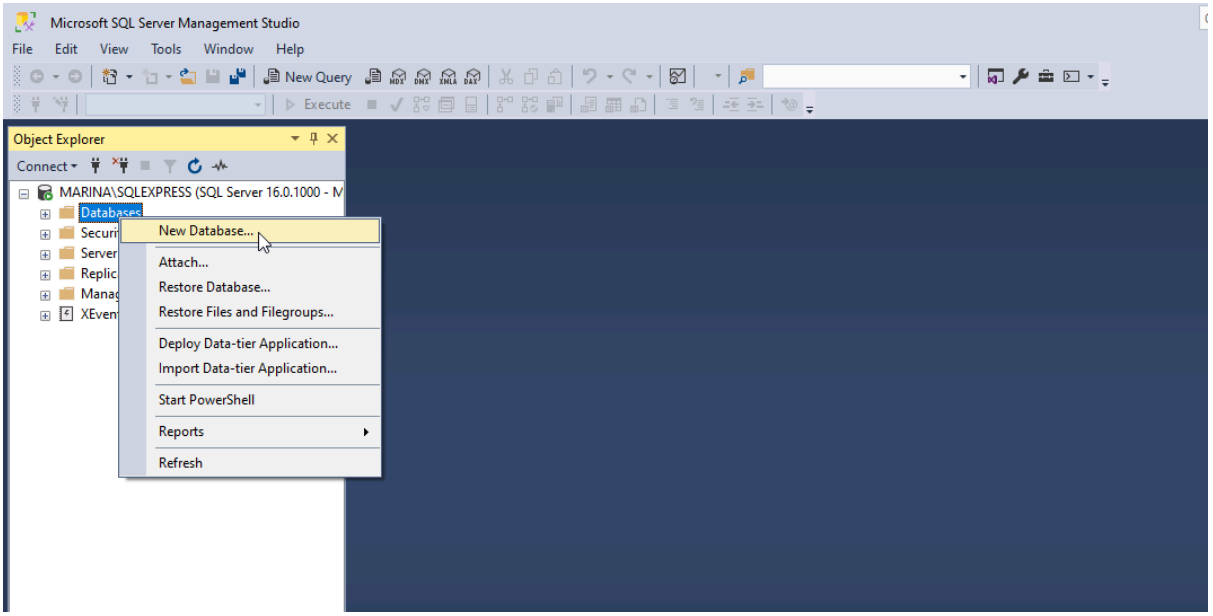
Creación de base de datos

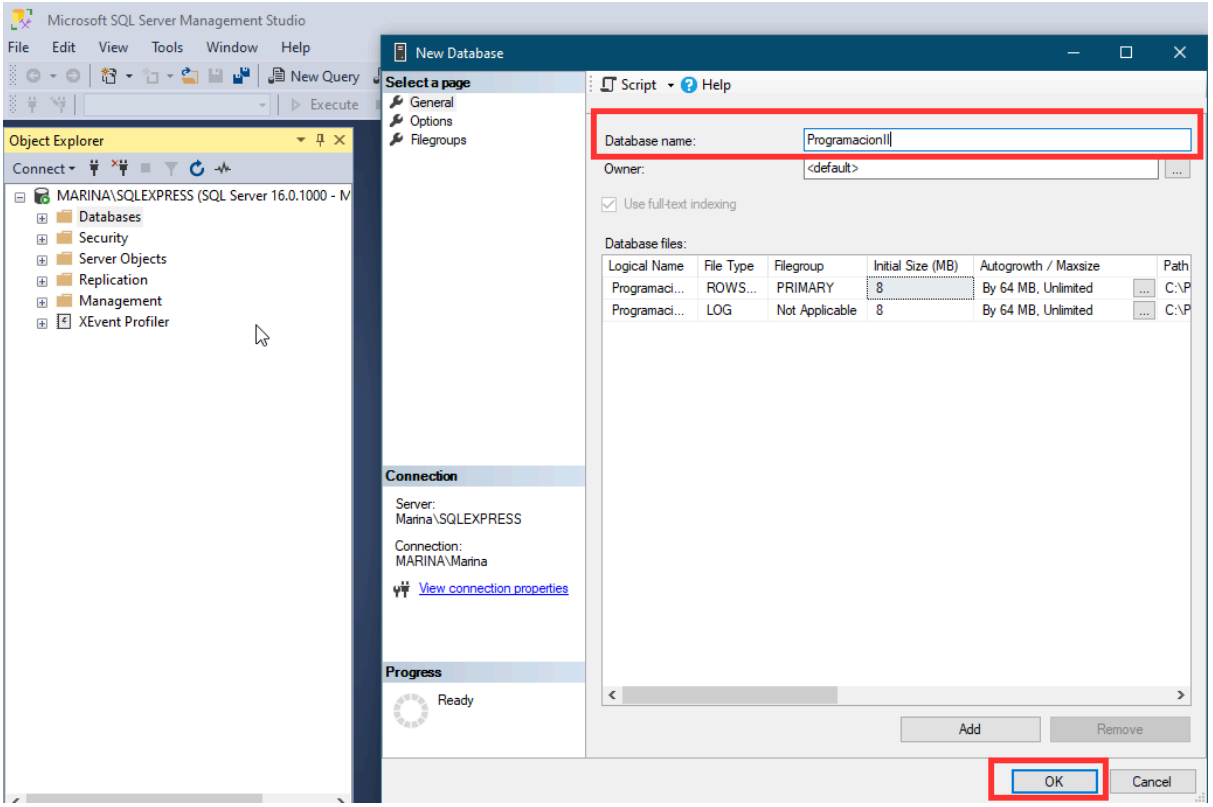
Conectado al servidor de SQL Server donde queremos crear la base de datos, en el **Explorador de objetos** (Object Explorer), haz clic derecho sobre el nodo **Databases** Selecciona New Database (Nueva base de datos).

En la ventana emergente **New Database**:

- **Database Name**: Escribe el nombre de la base de datos que deseas crear.
- Opcionalmente, puedes configurar los archivos de la base de datos (ubicación, tamaño, etc.) en la pestaña **Files** si necesitas ajustes personalizados.

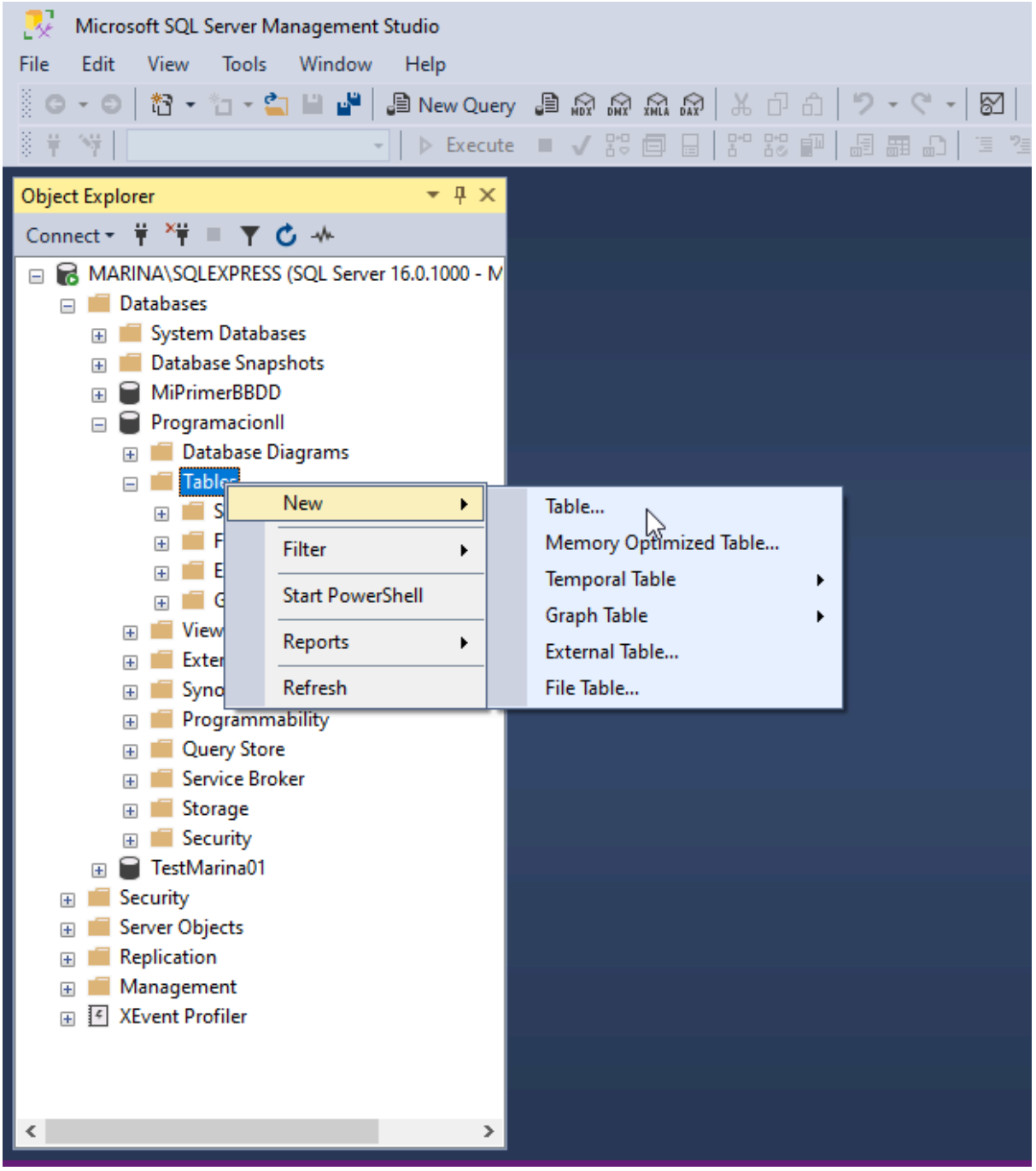
La nueva base de datos aparecerá en el nodo **Databases** en el **Explorador de objetos**.


Microsoft SQL Server Management Studio
File Edit View Tools Window Help
New Query
Execute
Object Explorer
Connect
MARINA\SQLEXPRESS (SQL Server 16.0.1000 - M
Databases
New Database...
Attach...
Restore Database...
Restore Files and Filegroups...
Deploy Data-tier Application...
Import Data-tier Application...
Start PowerShell
Reports
Refresh


Microsoft SQL Server Management Studio
File Edit View Tools Window Help
New Query
Execute
Object Explorer
Connect
MARINA\SQLEXPRESS (SQL Server 16.0.1000 - M
Databases
Security
Server Objects
Replication
Management
XEvent Profiler
New Database
Select a page
General
Options
Filegroups
Script
Help
Database name:
ProgramacionII
Owner:
<default>
Use full-text indexing
Database files:
Logical Name
File Type
Filegroup
Initial Size (MB)
Autogrowth / Maxsize
Path
Programaci...
ROWS...
PRIMARY
8
By 64 MB, Unlimited
Programaci...
LOG
Not Applicable
8
By 64 MB, Unlimited
Connection
Server:
Marina\SQLEXPRESS
Connection:
MARINA\Marina
View connection properties
Progress
Ready
Add
Remove
OK
Cancel

Creación de tablas:

En el **Explorador de objetos**, expandimos el servidor y luego el nodo de **Bases de datos (Databases)**.
Seleccionamos la base de datos en la que deseamos crear la nueva tabla.



Establecer clave primaria:

Abrir la tabla:

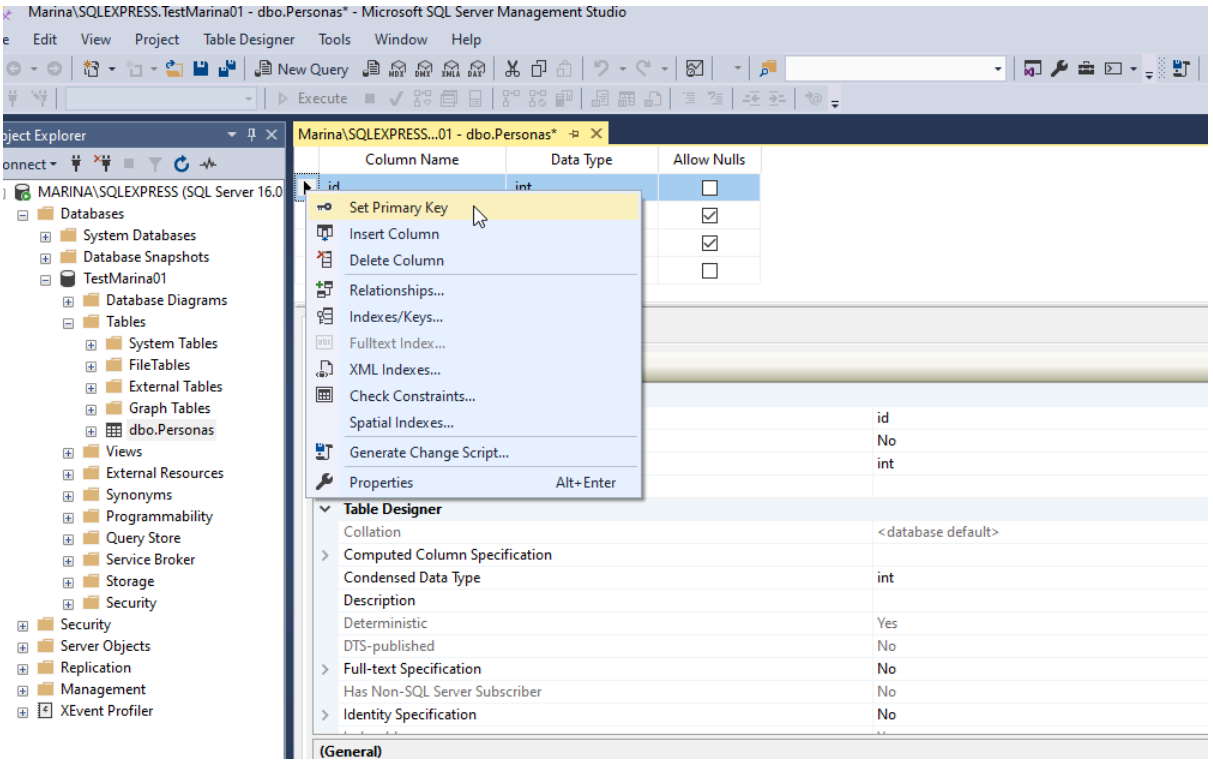
- En el **Explorador de Objetos**, encuentra tu base de datos y despliega el árbol para encontrar la tabla donde quieres agregar la clave primaria.
- Haz clic derecho en la tabla y selecciona **Diseño**.

Seleccionar columna(s):

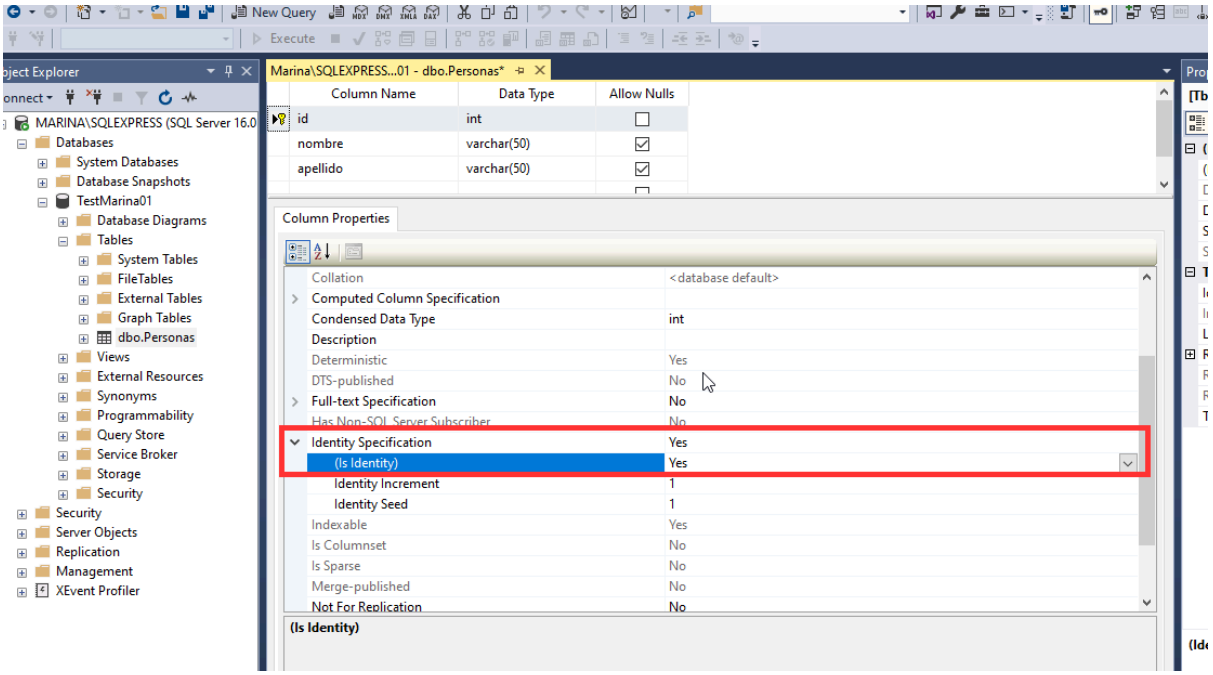
- En el diseñador de tablas, selecciona la columna o las columnas que deseas definir como clave primaria.
- Puedes mantener presionada la tecla `Ctrl` para seleccionar varias columnas si quieres una clave primaria compuesta.

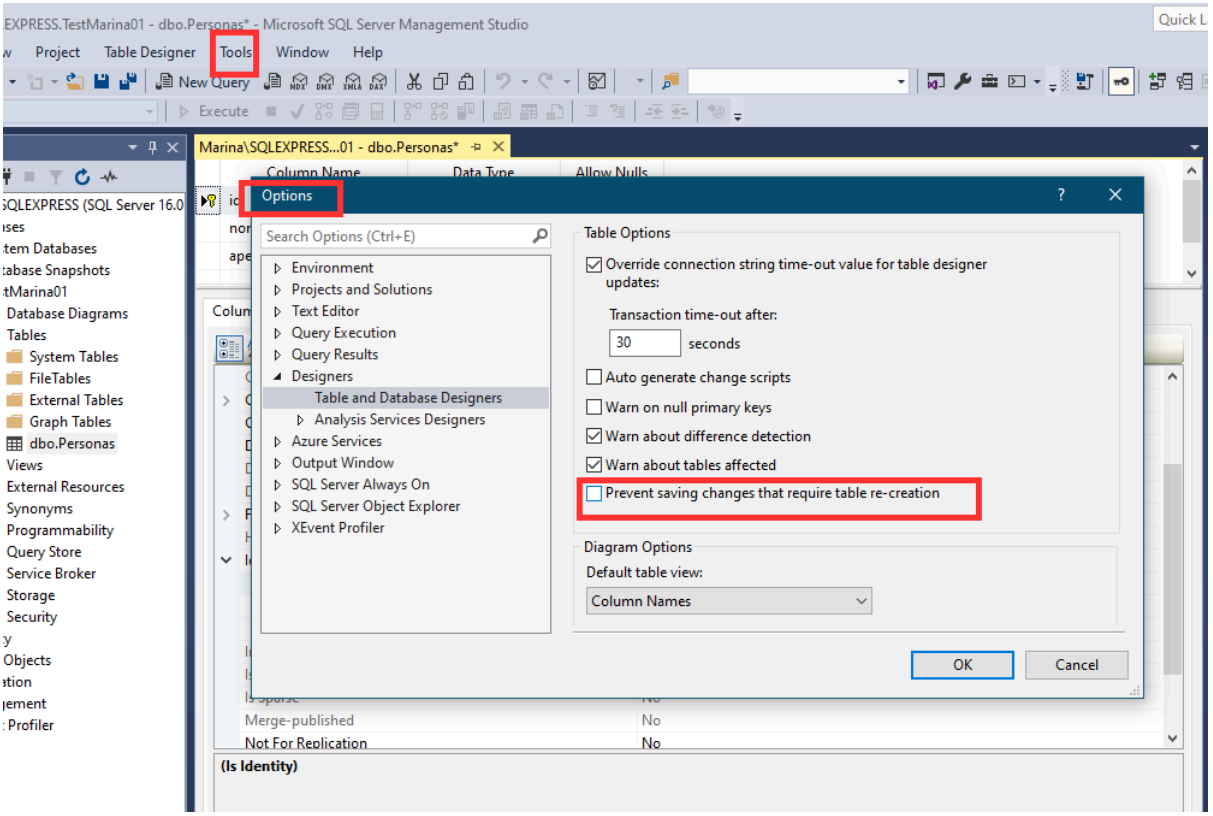
Definir como clave primaria:

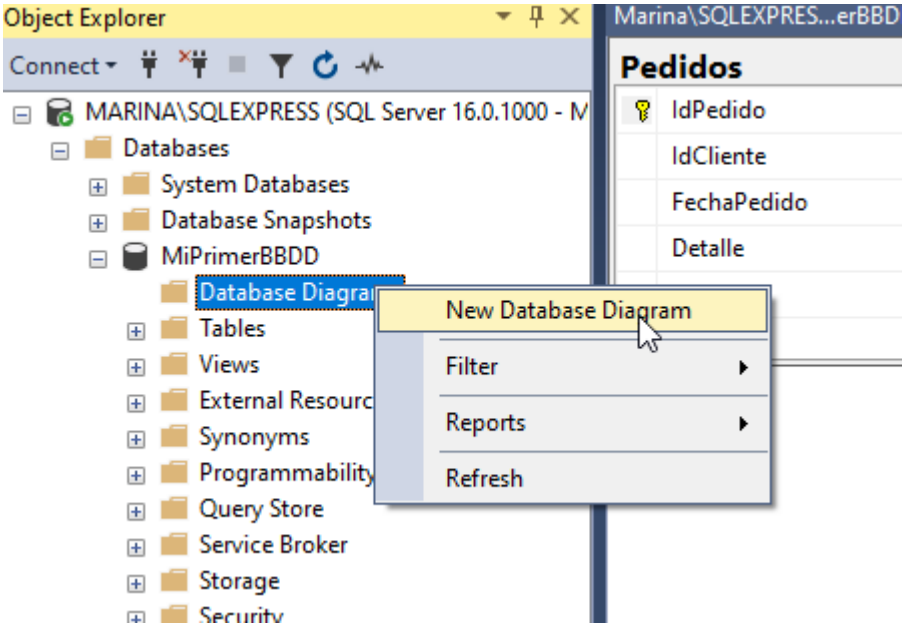
- Haz clic derecho sobre la columna seleccionada y elige la opción **Establecer clave primaria** del menú contextual.

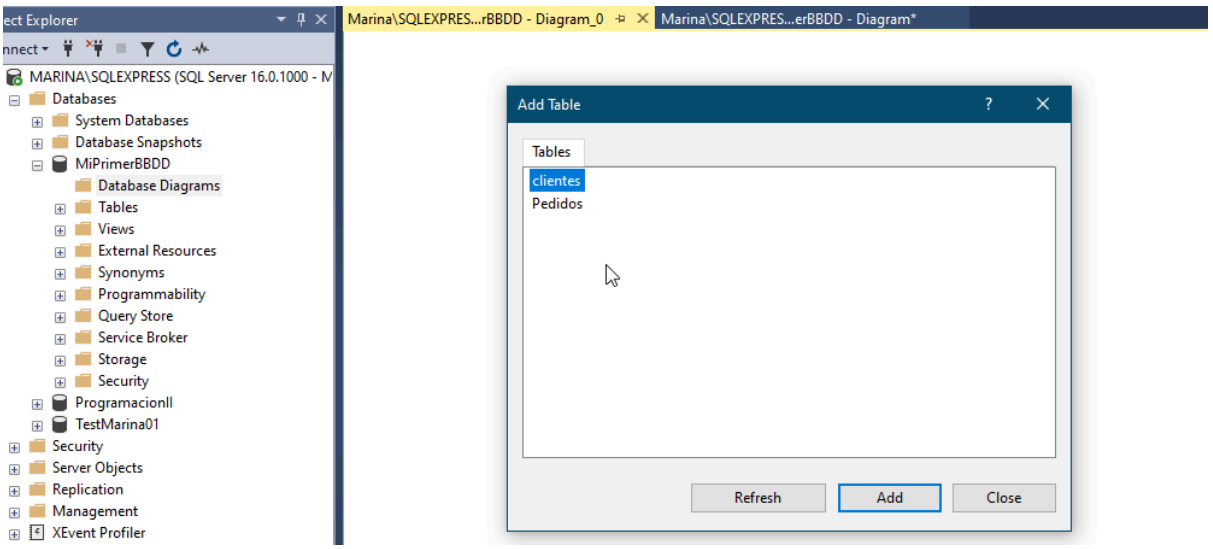


Campo autonumérico:



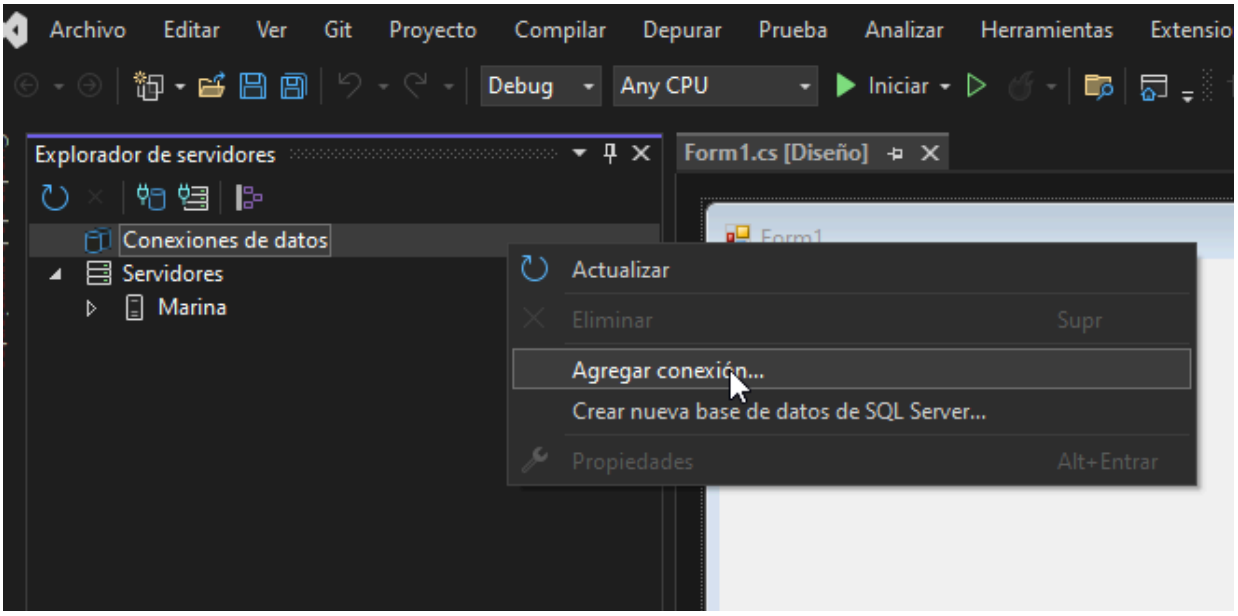

EXPRESS.TestMarina01 - dbo.Personas* - Microsoft SQL Server Management Studio
Project
Table Designer
Tools
Window
Help
New Query
Execute
Marina\SQLEXPRESS...01 - dbo.Personas*
Column Name
Data Type
Allow Nulls
Options
Search Options (Ctrl+E)
Environment
Projects and Solutions
Text Editor
Query Execution
Query Results
Designers
Table and Database Designers
Analysis Services Designers
Azure Services
Output Window
SQL Server Always On
SQL Server Object Explorer
XEvent Profiler
Table Options
Override connection string time-out value for table designer updates:
Transaction time-out after:
30
seconds
Auto generate change scripts
Warn on null primary keys
Warn about difference detection
Warn about tables affected
Prevent saving changes that require table re-creation
Diagram Options
Default table view:
Column Names
OK
Cancel
Merge-published
No
Not For Replication
No
(Is Identity)
Database Diagrams
Tables
System Tables
FileTables
External Tables
Graph Tables
dbo.Personas
Views
External Resources
Synonyms
Programmability
Query Store
Service Broker
Storage
Security

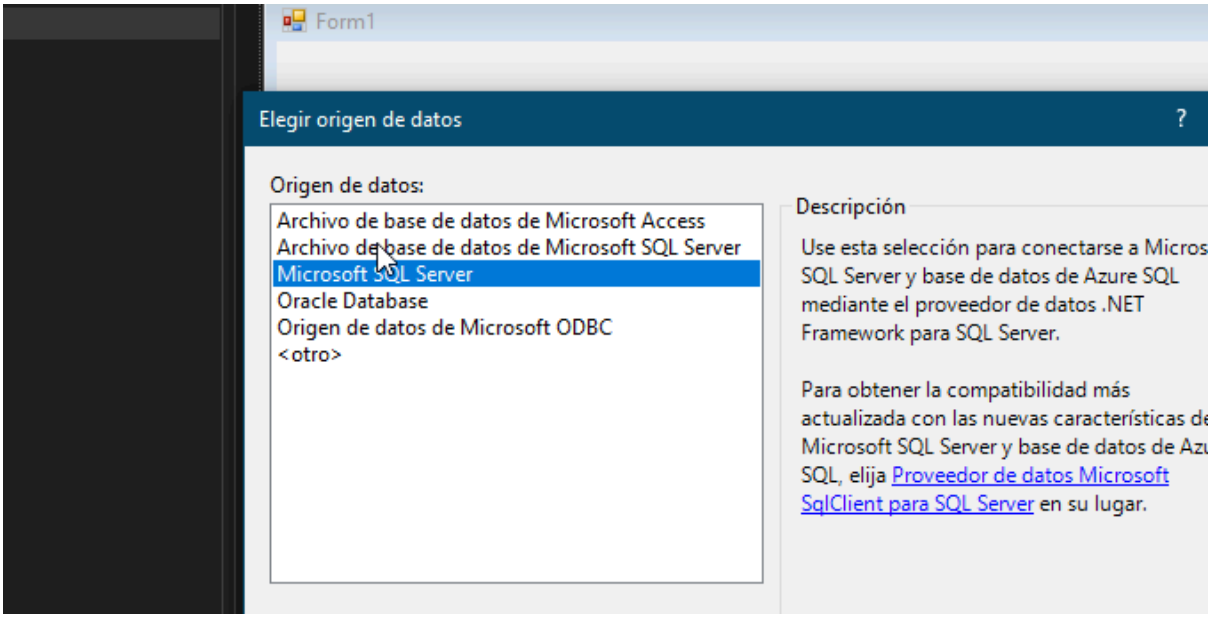

Object Explorer
Connect
MARINA\SQLEXPRESS (SQL Server 16.0.1000 - M
Databases
System Databases
Database Snapshots
MiPrimerBBDD
Database Diagrams
Tables
Views
External Resources
Synonyms
Programmability
Query Store
Service Broker
Storage
Security
New Database Diagram
Filter
Reports
Refresh
Marina\SQLEXPRES...erBBD
Pedidos
IdPedido
IdCliente
FechaPedido
Detalle


ect Explorer
nnect
MARINA\SQLEXPRESS (SQL Server 16.0.1000 - M
Databases
System Databases
Database Snapshots
MiPrimerBBDD
Database Diagrams
Tables
Views
External Resources
Synonyms
Programmability
Query Store
Service Broker
Storage
Security
ProgramacionII
TestMarina01
Security
Server Objects
Replication
Management
XEvent Profiler
Marina\SQLEXPRES...rBBDD - Diagram_0
Marina\SQLEXPRES...erBBDD - Diagram*
Add Table
Tables
clientes
Pedidos
Refresh
Add
Close

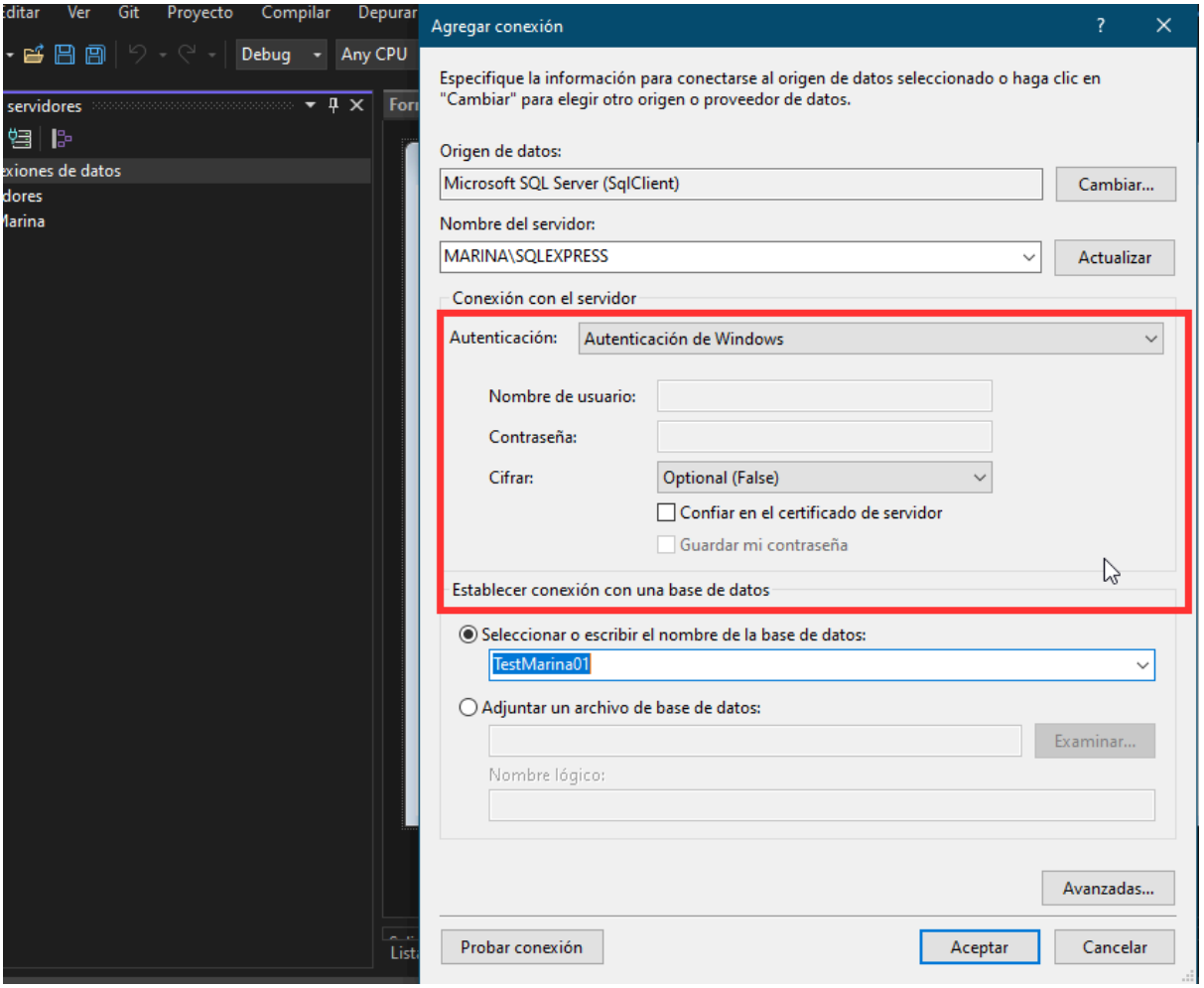
Conectar la base de datos con Visual Studio

Abrir Explorador de servidores: Menú Ver → Explorador de servidores y agregamos una nueva conexión haciendo clic derecho sobre conexiones de base de datos.

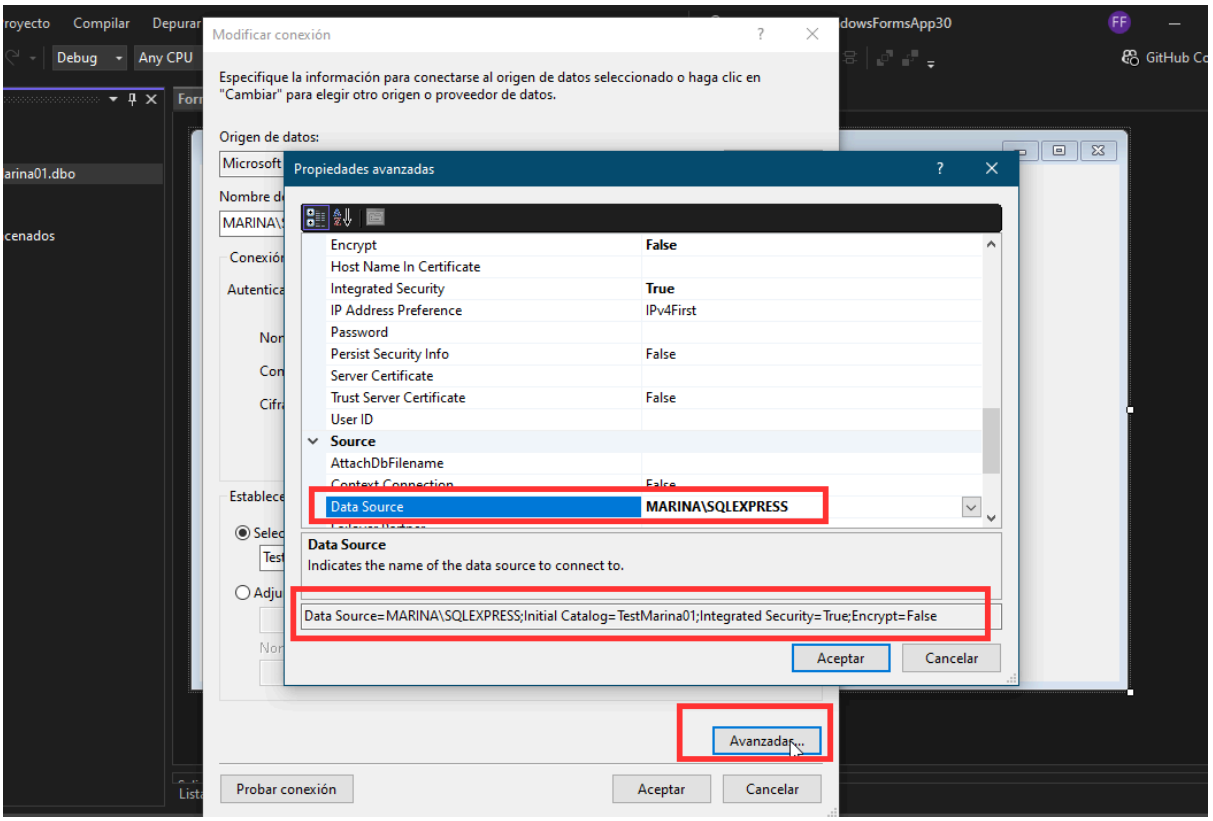




Colocamos el nombre del servidor (Server Name) y luego el nombre de la base de datos a la cual nos vamos a conectar.


Editar
Ver
Git
Proyecto
Compilar
Depurar
Debug
Any CPU
servidores
xiones de datos
dores
Marina
Agregar conexión
Especifique la información para conectarse al origen de datos seleccionado o haga clic en "Cambiar" para elegir otro origen o proveedor de datos.
Origen de datos:
Microsoft SQL Server (SqlClient)
Cambiar...
Nombre del servidor:
MARINA\SQLEXPRESS
Actualizar
Conexión con el servidor
Autenticación:
Autenticación de Windows
Nombre de usuario:
Contraseña:
Cifrar:
Optional (False)
Confiar en el certificado de servidor
Guardar mi contraseña
Establecer conexión con una base de datos
Seleccionar o escribir el nombre de la base de datos:
TestMarina01
Adjuntar un archivo de base de datos:
Examinar...
Nombre lógico:
Avanzadas...
Probar conexión
Aceptar
Cancelar

Cadena de conexión:



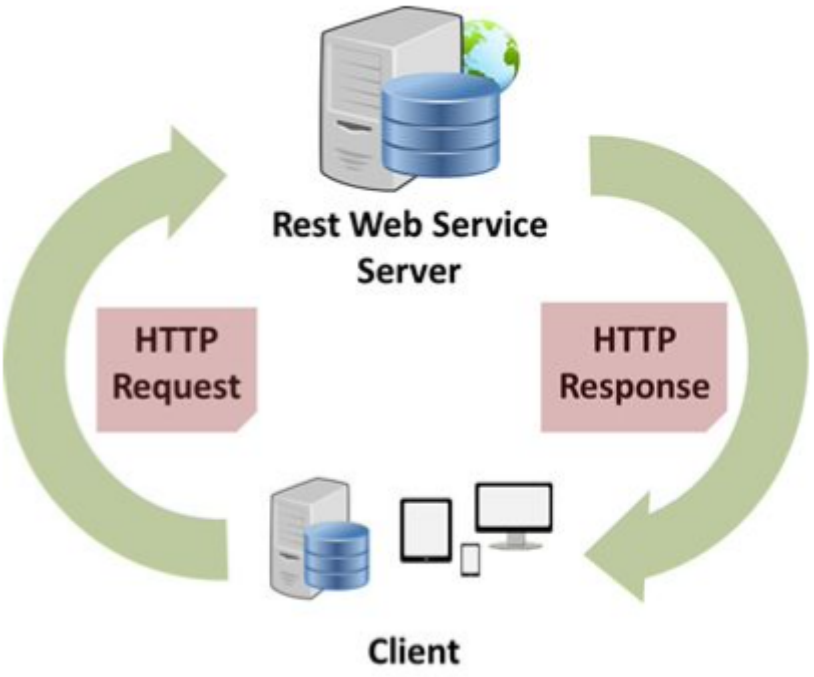
Introducción a Servicios Web

En la actualidad, los servicios web se han convertido en un componente esencial de la arquitectura de Internet, facilitando la comunicación y la interacción entre diversas aplicaciones y sistemas. Estos servicios permiten que diferentes plataformas se conecten y compartan datos de manera eficiente, independientemente de su ubicación o tecnología subyacente. Utilizando protocolos estándar como **HTTP, XML y JSON**, los servicios web permiten la integración de aplicaciones, la automatización de procesos y la mejora de la colaboración entre empresas. A medida que las empresas buscan optimizar sus operaciones y mejorar la experiencia del usuario, entender el funcionamiento y las aplicaciones de los servicios web se vuelve fundamental.

Antes de adentrarnos en las tecnologías, debemos conocer el ecosistema digital básico. Cualquier sistema que se conecte a Internet, lo llamaremos **Cliente**. Este puede ser el navegador web de tu celular o computadora, sistemas de pago, aplicaciones, etc.
La dinámica básica consiste en la arquitectura **REST** donde un **Cliente** envía **Peticiones (Request)** a un **Servidor** y este le devuelve una **Respuesta (Response)**.

REST

REST o Representational State Transfer es un estilo de Arquitectura a la hora de realizar una comunicación entre cliente y servidor.



Para poder lograrlo, lo hace a través de la dirección del Servidor pudiendo ser de dos maneras:

- **Dirección IP (Internet Protocol):** es la identificación única mediante la cual el Servidor se conecta a Internet.
- **Nombre de dominio (DNS - Domain Name Server)**: El servidor de DNS será encargado de traducir ese nombre de dominio en una dirección IP para poder conectarnos con el Servidor final y así enviarle las peticiones. Por ejemplo "google.com".

REST - Características

- Se apoya sobre el protocolo **HTTP**.
- Crea una solicitud **HTTP** que contiene toda la información necesaria, es decir, un **Request** a un servidor tiene toda la información necesaria y solo espera una **Response**, ósea una respuesta en concreto.
- Se apoya en los métodos básicos de **HTTP** (verbos **HTTP**), como son:
    - **Post**: Para crear recursos nuevos.
    - **Get**: Para obtener un listado o un recurso en concreto.
    - **Put**: Para modificar un recurso.
    - **Patch**: Para modificar un recurso (en forma parcial).
    - **Delete**: Para borrar un recurso.
- Todos los objetos se manipulan mediante **URI** (identificador uniforme de recursos)
- Utiliza como formato de intercambio **JSON** / **XML**.
    - **JSON** es un formato ligero de intercambio de datos.
    - Leerlo y escribirlo es simple para humanos, mientras que para las máquinas es simple interpretarlo y generarlo.

Una web útil para practicar solicitudes es la API pública de https://reqres.in/

API

Una interfaz de programación de aplicaciones (Application Programming Interface) es un conjunto de herramientas, definiciones y protocolos que se usan para diseñar e integrar aplicaciones. Permite que un producto o servicio se comunique con otros productos y servicios sin necesidad de conocer su implementación interna. Por ejemplo, cuando usas una aplicación de clima en tu teléfono, ésta puede utilizar una API para obtener datos meteorológicos de un servidor externo.

APIs No Web

1. **API de Sistema Operativo**:
    - **Windows API (WinAPI)**: Interacción con el sistema operativo Windows para crear aplicaciones de escritorio.
    - **POSIX API**: Interfaces de programación para sistemas operativos similares a Unix.
2. **API de Bases de Datos**:

- **ODBC (Open Database Connectivity)**: Conexión entre aplicaciones y diversas bases de datos.
- **JDBC (Java Database Connectivity)**: Conexión de aplicaciones Java con bases de datos relacionales.

3. **API de Hardware**:
    - **OpenGL**: Renderizado de gráficos 2D y 3D, común en aplicaciones de gráficos y juegos.
    - **DirectX**: API de Microsoft para crear videojuegos y multimedia en plataformas Windows.
    - **Centralita de Autos (ECU)**:
        i. **OBD-II (On-Board Diagnostics)**: Un estándar de diagnóstico utilizado en vehículos que permite acceder a datos del sistema del motor y otros componentes. Las APIs para OBD-II permiten a los desarrolladores crear aplicaciones que se comunican con el vehículo para leer datos como la velocidad, las RPM del motor y los códigos de error.
    - **Bluetooth**:
        i. **Bluetooth API**: Las plataformas móviles (Android, iOS) y algunas bibliotecas de escritorio ofrecen APIs para interactuar con dispositivos Bluetooth. Estas APIs permiten la conexión, comunicación y transferencia de datos entre dispositivos, como teléfonos, auriculares, y dispositivos IoT.
        ii. **Android Bluetooth API**: Permite a los desarrolladores crear aplicaciones que se conectan a dispositivos Bluetooth, gestionar conexiones, y transferir datos.
        iii. **Core Bluetooth para iOS**: Similar al de Android, esta API permite la interacción con dispositivos Bluetooth Low Energy (BLE).
    - **Módem/Router**:
        i. **UPnP (Universal Plug and Play)**: Permite la comunicación y control entre dispositivos en una red local. Muchas configuraciones de módems/routers ofrecen APIs para administrar configuraciones de red, puertos y otros parámetros.
        ii. **SNMP (Simple Network Management Protocol)**: Utilizado para la administración de dispositivos en redes IP, permite monitorear y configurar routers y switches.
    - **IoT (Internet de las Cosas)**:
        i. **MQTT**: Un protocolo de mensajería ligero diseñado para la comunicación entre dispositivos IoT, a menudo utilizados en sistemas de automatización del hogar.
        ii. **CoAP (Constrained Application Protocol)**: Un protocolo para la comunicación de dispositivos IoT en redes restringidas.
    - **Cámaras y Dispositivos de Video**:
        i. **RTSP (Real Time Streaming Protocol)**: Utilizado para transmitir video y audio en tiempo real desde cámaras IP y otros dispositivos de video.
4. **API de Dispositivos Móviles**:
    - **Android API**: Interacción con funciones del sistema operativo Android para aplicaciones móviles.

    - **iOS SDK**: APIs para crear aplicaciones en dispositivos iOS.
5. **API de Software de Aplicación**:
    - **API de Adobe Photoshop**: Para crear complementos y automatizar tareas en Photoshop.
    - **API de Microsoft Office**: Automatización y creación de complementos para Word y Excel.
6. **API de Mensajería**:
    - **MQTT (Message Queuing Telemetry Transport)**: Protocolo de mensajería ligero utilizado en IoT.
    - **AMQP (Advanced Message Queuing Protocol)**: Protocolo abierto para mensajería entre aplicaciones.

APIs Web (Arquitectura REST)

1. **API de Google Maps**: Permite integrar mapas, direcciones y lugares en aplicaciones web y móviles.
2. **API de Twitter**: Facilita la interacción con Twitter para enviar tweets, seguir usuarios y más.
3. **API de GitHub**: Permite acceder y manipular datos de repositorios y usuarios en GitHub.
4. **API de Spotify**: Ofrece acceso a información sobre música, álbumes y listas de reproducción.
5. **API de OpenWeather**: Proporciona datos meteorológicos en tiempo real y pronósticos.
6. **API de Stripe**: Permite procesar pagos y gestionar transacciones en línea.

API REST

Es una API que implementa parcialmente la arquitectura REST, utilizando algunos verbos HTTP como GET. Por ejemplo, al solicitar información del clima.

API RESTful

En este caso, se garantiza la implementación de toda la arquitectura REST.

API REST vs API RESTful

Característica	API REST	API RESTful
Definición	Interfaz que sigue parcialmente los principios de REST.	Implementa completamente los principios de REST.
Verbos HTTP	Puede usar solo algunos verbos HTTP (GET, POST).	Utiliza todos los verbos HTTP (GET, POST, PUT, DELETE, PATCH).
Stateless	Puede ser stateful (mantener estado) en algunas implementaciones.	Siempre stateless (no mantiene estado entre solicitudes).
Recursos	Puede no representar recursos de manera óptima.	Cada recurso es representado claramente y puede ser manipulado a través de su URI.
Formato de Datos	No se especifica un formato estándar.	Suele utilizar formatos estándar como JSON o XML.
Caching	Puede o no soportar caching.	Soporta caching, lo que mejora el rendimiento.
Interacción	La interacción puede ser más limitada.	Permite interacciones completas y flexibles con recursos.
HATEOAS	Generalmente no incluye HATEOAS (Hypermedia as the Engine of Application State).	Se espera que soporte HATEOAS, proporcionando enlaces a recursos relacionados.

En resumen:

- **API REST** es un término más amplio que se refiere a una API que sigue algunos principios de REST, pero puede no implementarlos todos.
- **API RESTful** se refiere a una implementación completa y correcta de REST, siguiendo todos sus principios y características.

Existen diversas webs para practicar el consumo de una API RESTFul, en el siguiente link existen distintos endpoints con su documentación https://reqres.in/.

Servidor

Un servidor web es un software y un hardware que utiliza el protocolo HTTP (Hypertext Transfer Protocol) y otros protocolos para responder a las peticiones de los clientes realizadas a través de la World Wide Web (Internet) o una red privada (Intranet).

- Pueden ser propios o alquilados.
- Servidores domésticos o profesionales (datacenter).
- Pueden tener programas en ejecución (corriendo) constantemente.
- Puede tener gestor de cuentas de Hosting o no.

Hosting

Dentro de los **Servidores**, podemos fragmentar esta infraestructura en diferentes cuentas de usuarios. Una cuenta de hosting es un alquiler parcial de un servidor, esto es ideal para proyectos pequeños y medianos que no requieran alto tráfico de peticiones-respuestas.

Características básicas:

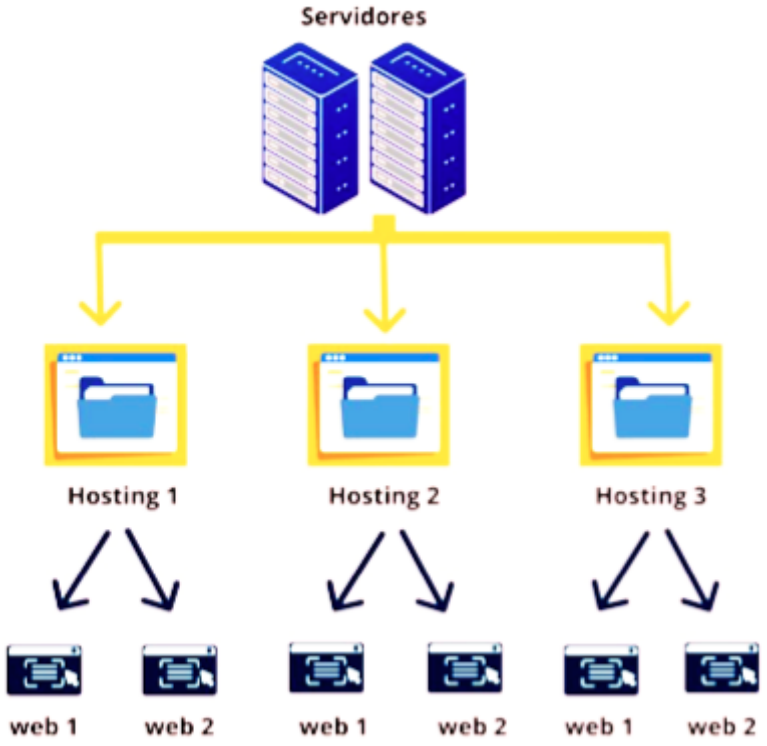
- Son de alquiler mensual o anual.
- Puede gestionar una o más **Cuentas de dominio**.
- Reciben las peticiones del servidor de DNS para ofrecer archivos de una carpeta específica.
- Pueden o no tener programas corriendo, como por ejemplo, aplicaciones NodeJS.

Cuentas de dominio

Dentro de nuestro plan de **Hosting**, podemos encontrar una o varias Cuentas de dominio. Estas son las encargadas de gestionar el dominio donde publicaremos nuestro proyecto. Si tenemos un proyecto web o API, en ellas podremos configurar el dominio y las características de publicación.

Aquí se configura todo lo referido a un sitio web:

- Cuentas de correo.
- Base de Datos.
- Redirecciones.
- Permisos de carpetas.
- Permisos de usuario.
- Certificados SSL.



Certificado SSL

Dentro de nuestra **Cuenta de dominio**, tenemos la posibilidad de instalar un Certificado SSL. Este certificado tiene la capacidad de encriptar la información enviada de punto a punto, quiere decir que las Peticiones y Respuestas, viajarán encriptadas bajo el sello de nuestro certificado obteniendo así, una encriptación personalizada para nuestro dominio. Un certificado no solo protege las conexiones, sino que también aporta reputación al sitio. A fines de la década pasada, por cuestiones de seguridad, los navegadores exigen tener un certificado para minimizar los ciberdelitos asociados a la vulnerabilidad de las conexiones no seguras.

Secure Sockets Layer

- Permiten navegar de forma segura.
- Encripta las conexiones de punto a punto. Desde el servidor, hasta el navegador.
- Habilita la redirección HTTPS.
- Google Chrome y Mozilla Firefox, entre otros navegadores, marcan como inseguros a los sitios que no poseen SSL.

Protocolos de conexión

Los protocolos de conexión son fundamentales en la comunicación entre dispositivos en una red. Estos conjuntos de reglas y estándares permiten que diferentes sistemas se entiendan y trabajen juntos, garantizando una transferencia de datos eficiente y segura.

Existen diversos protocolos, cada uno diseñado para cumplir funciones específicas. Por ejemplo, el HTTP (Hypertext Transfer Protocol) es el protocolo que permite la transferencia de páginas web, mientras que el FTP (File Transfer Protocol) se utiliza para la transferencia de archivos entre computadoras.

Además, los protocolos de conexión pueden clasificarse en dos categorías principales:

Protocolos orientados a conexión: Establecen una conexión antes de que se inicie la transferencia de datos. Esto asegura que ambos extremos estén listos para comunicarse, como es el caso de TCP (Transmission Control Protocol), que garantiza la entrega de datos y el orden correcto de las transmisiones.

Protocolos sin conexión: Permiten la transferencia de datos sin establecer una conexión previa. Esto puede resultar en un menor consumo de recursos, aunque puede implicar riesgos como la pérdida de datos, como ocurre con UDP (User Datagram Protocol).

El correcto uso y comprensión de los protocolos de conexión es esencial para garantizar una comunicación fluida y segura en el entorno digital. A medida que la tecnología avanza, estos protocolos continúan evolucionando, adaptándose a las nuevas necesidades de seguridad y eficiencia en la transmisión de información.

Un puerto es un punto de conexión virtual que permite al Servidor y al Cliente enviar y recibir datos a través de una red. Cada puerto se identifica con un número que va desde el 0 hasta el 65545. Cada uno tiene una funcionalidad específica y algunos son utilizados por los Protocolos para enviar y recibir información.

Protocolo	**Descripción**	**Puerto**
HTTP	Hypertext Transfer Protocol: transferencia de páginas web.	80
HTTPS	Hypertext Transfer Protocol Secure: versión segura de HTTP.	443
FTP	File Transfer Protocol: transferencia de archivos.	21 - Control 20 - Datos
SFTP	Secure File Transfer Protocol: versión segura de FTP que utiliza SSH.	22
SMTP	Simple Mail Transfer Protocol: envío de correos electrónicos.	25 587 - Segura

IMAP	Internet Message Access Protocol: gestiona correos en un servidor.	143 993 - Segura
POP3	Post Office Protocol versión 3:	110 995 - Segura
TCP	Transmission Control Protocol: orientado a conexión que garantiza la entrega.	Varios
UDP	User Datagram Protocol: sin conexión que permite transmisión rápida.	Varios
DNS	Domain Name Server: resolución de nombres de dominio.	53
DHCP	Dynamic Host Configuration Protocol: asignar dinámicamente direcciones IP.	67 - Servidor 68 - Cliente
SNMP	Simple Network Management Protocol: gestión y monitoreo de dispositivos en red.	161 - Com. 162 - Traps

Git (gonza)

Git es un herramienta de control de versiones (o sistema de versionado) creado por Linus Torvalds (también creador del kernel Linux).

Una herramienta de control de versiones lleva adelante la gestión de los diversos cambios que se realizan sobre los elementos de un repositorio. Permite trabajar en distintas ramas a través de un mismo proyecto, donde por ejemplo, las más conocidas son ‘master’ o ‘main’ para la rama principal y ‘dev’ para la rama de desarrollo.

El objetivo es gestionar los cambios en el código de todo el proyecto (repositorio), donde estaremos guardandolos en paquetes llamados ‘commits’. De esta forma, cada grupo o paquete de cambios (archivos con cambios), quedan documentados en una línea de tiempo de desarrollo, pudiendo volver hacia esa versión para ver el código antiguo o volviendo todo el proyecto a ese punto.

Por esto decimos que Git es un software de Control de Versiones, ya que cada paquete (commit) que estamos publicando en la rama, genera una nueva versión del proyecto.

¿Para qué se usa?

- Compartir código fuente con otras personas.
- Tener un historial de los cambios realizados en el código.
- Saber quién los hizo y cuándo.
- Resolver los conflictos que surjan cuando dos o más personas modifiquen el mismo archivo (merge).

Repositorios

- Un repositorio es un espacio (local o remoto) que tenemos asignado para poder alojar todos los archivos de nuestro proyecto.
- Un repositorio es el lugar donde van a estar todos los commits que forman parte del historial del proyecto.

Repositorios locales y remotos

- GIT trabaja con un repositorio local que está en nuestro equipo, donde iremos agregando nuestros commits.
- También trabaja con uno remoto en el cual podemos subir nuestros commits o del cual podemos bajarnos los commits que haya subido alguien.
  De esta manera, cada integrante nuestro proyecto, estará trabajando con su repositorio local hasta que decida subir sus commits al repositorio remoto. El resto de los miembros se conectarán desde sus equipos para descargar los commits nuevos subidos por sus compañeros. El repositorio remoto será el encargado de almacenar la versión más actualizada y oficial para todo el equipo.

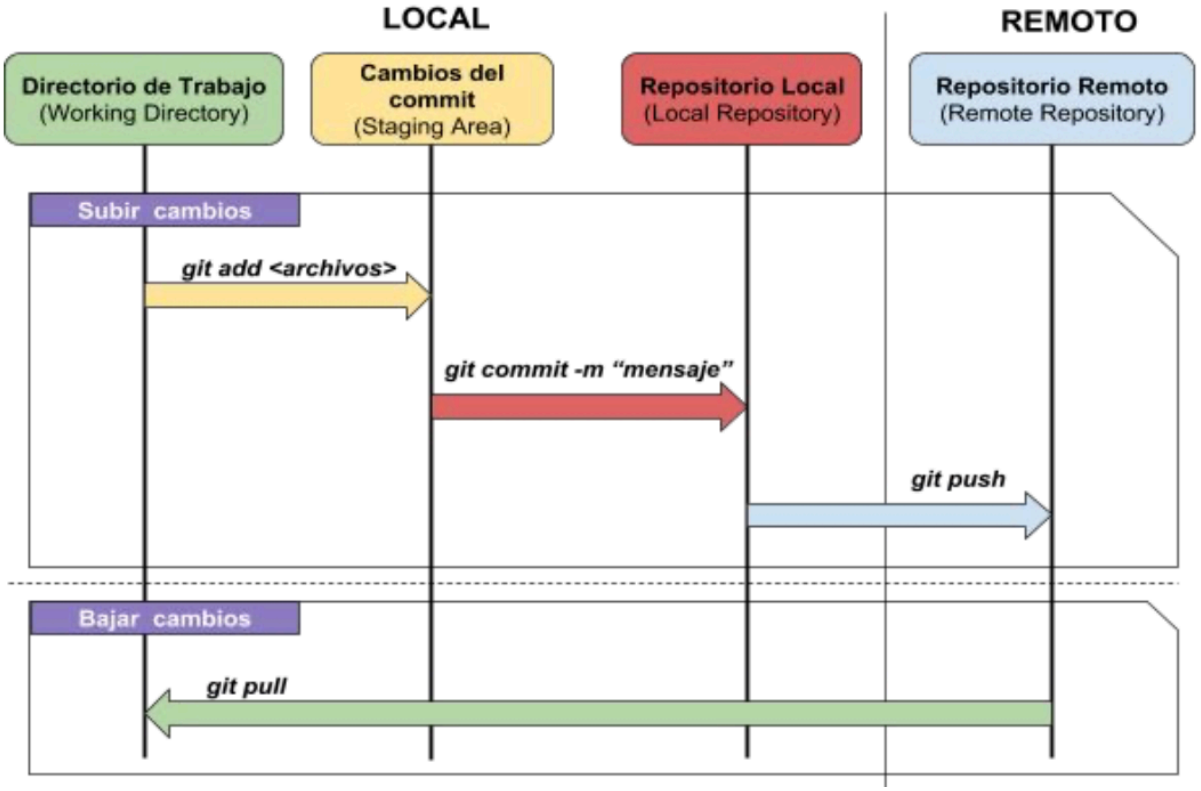
Commit

Es un paquete de cambios que contiene solamente los cambios que le hemos realizado a nuestros archivos, así como los nuevos archivos y los eliminados de nuestro proyecto. A este paquete, le podemos adjuntar una descripción para saber qué contiene.

¿Cómo se usa? (Alto nivel)

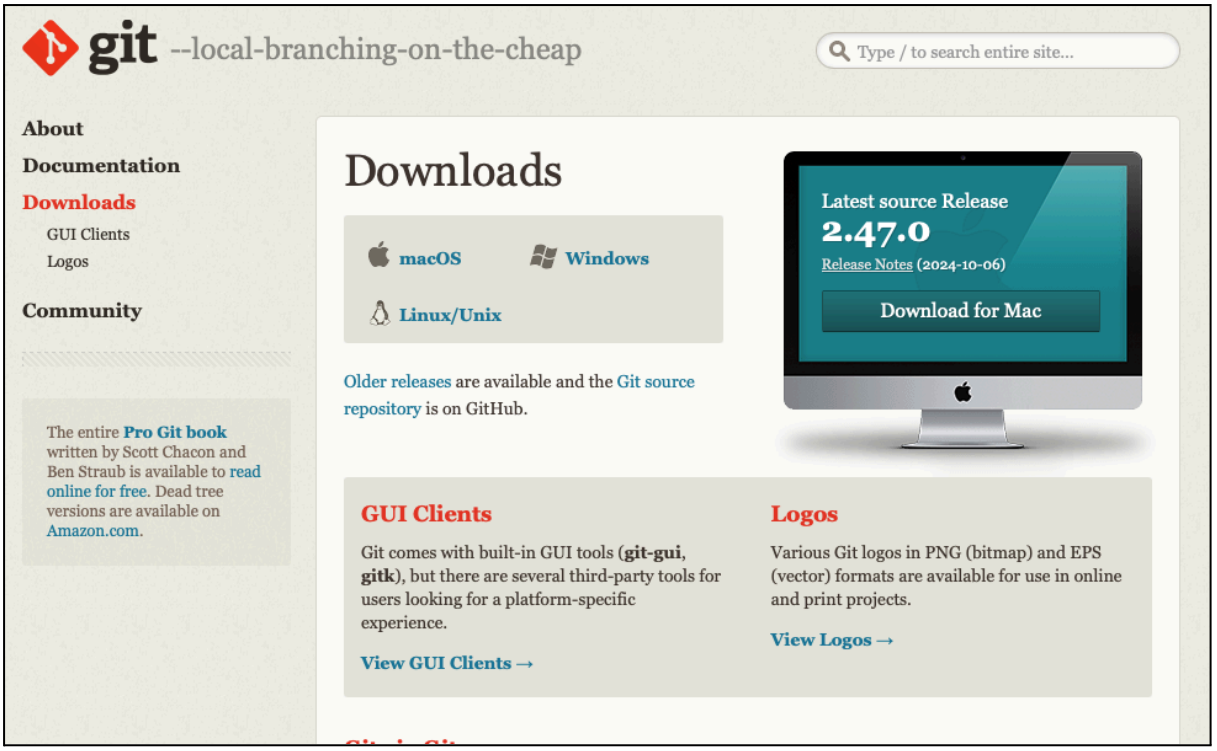
1. Creamos/borramos/modificamos archivos en una carpeta asociada a un repositorio (localmente, en nuestro S.O).
2. Seleccionamos los archivos que van a ser parte de un commit.
3. Confirmamos el commit para agregarlo al repositorio local.
4. Subimos los commits de nuestro repositorio local al repositorio remoto.

Cada commit determina una versión específica del proyecto. Cada commit tiene un identificador único (un hash) y captura el estado completo del código en ese momento, junto con un mensaje que describe los cambios realizados.



Instalación

GIT es una herramienta CLI, se ejecuta desde la terminal de linea de comandos. Para poder utilizarlo en nuestra computadora, debemos instalarlo previamente desde https://git-scm.com/downloads .



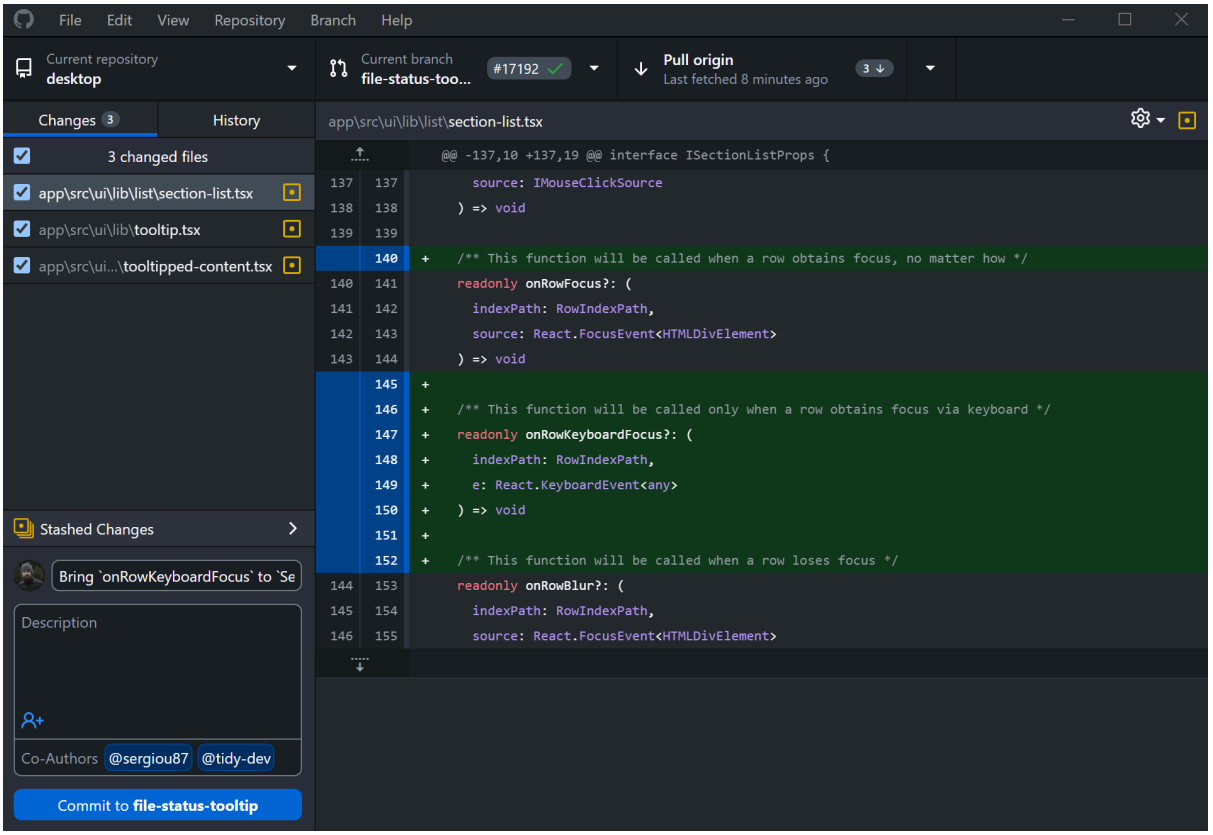
GitHub Web y Desktop

GitHub es una plataforma de desarrollo colaborativo basada en la web que utiliza Git como sistema de control de versiones. Permite a los desarrolladores almacenar, gestionar y colaborar en proyectos de código fuente. Algunas de sus características clave incluyen:

- **Almacenamiento de Repositorios**: GitHub ofrece un espacio para alojar repositorios, tanto públicos como privados, permitiendo a los usuarios acceder y contribuir a proyectos desde cualquier lugar.
- **Colaboración:** Facilita la colaboración entre equipos y desarrolladores mediante herramientas como pull requests, que permiten revisar y discutir cambios antes de fusionarlos en la rama principal.
- **Control de Versiones:** Al estar basado en Git, GitHub permite a los usuarios gestionar el historial de cambios, realizar commits, crear ramas y resolver conflictos, todo desde una interfaz intuitiva.
- **Integración y Automatización:** Ofrece integraciones con diversas herramientas y servicios, permitiendo automatizar flujos de trabajo de desarrollo mediante GitHub Actions, que pueden realizar tareas como pruebas, despliegues y más.
- **Documentación y Proyectos:** Proporciona herramientas para documentar proyectos (como GitHub Pages) y gestionar tareas (con GitHub Projects), mejorando la organización y comunicación dentro de los equipos.
- **Comunidad y Open Source:** GitHub es un punto central para proyectos de código abierto, permitiendo a los desarrolladores compartir su trabajo y colaborar en iniciativas globales.

GitHub Desktop

GitHub se utiliza principalmente desde la web que es su primera interfaz gráfica. En la actualidad pueden gestionarse los repositorios desde aplicaciones móviles y también cuenta con una aplicación de escritorio que puede descargarse desde https://desktop.github.com/ .



Desde esta aplicación, podemos gestionar los commits locales y remotos de nuestro proyecto agilizando la conexión con GitHub.

Comandos Básicos de Git

- **Configuración**
  - **git config --global user.name "Tu Nombre"**: Configura tu nombre.
  - **git config --global user.email "tu.email@example.com"**: Configura tu correo electrónico.
- **Creación de Repositorios**
  - **git init**: Crea un nuevo repositorio vacío.
  - **git clone <url>**: Clona un repositorio remoto.
- **Estado y Seguimiento**
  - **git status**: Muestra el estado de los cambios en el repositorio.
  - **git add <archivo>**: Añade un archivo al área de preparación (staging).
  - **git add .** : Añade todos los archivos modificados al área de preparación.

    - **git add -a:** Añade todos los archivos modificados, nuevos y eliminados del área de preparación.
- **Confirmación de Cambios**
    - **git commit -m "Mensaje"**: Confirma los cambios en el área de preparación con un mensaje.
    - **git commit -a -m "Mensaje"**: Confirma todos los cambios de seguimiento y añade un mensaje.
- **Visualización de Cambios**
    - **git log**: Muestra el historial de commits.
    - **git diff**: Muestra las diferencias entre archivos no confirmados.
    - **git diff <commit1> <commit2>**: Muestra las diferencias entre dos commits.
- **Ramas**
    - **git branch**: Muestra las ramas existentes.
    - **git branch <nombre-rama>**: Crea una nueva rama.
    - **git checkout <nombre-rama>**: Cambia a la rama especificada.
    - **git checkout -b <nombre-rama>**: Crea y cambia a una nueva rama.
- **Fusión y Rebases**
    - **git merge <nombre-rama>**: Fusiona la rama especificada en la rama actual.
    - **git rebase <nombre-rama>**: Aplica los commits de la rama actual sobre la rama especificada.
- **Remotos**
    - **git remote -v**: Muestra las URLs de los repositorios remotos.
    - **git push <remoto> <rama>:** Envía los cambios de la rama local al repositorio remoto.
    - **git pull <remoto> <rama>**: Trae y fusiona los cambios del repositorio remoto en la rama actual.
- **Deshacer Cambios**
    - **git reset <archivo>**: Elimina un archivo del área de preparación.
    - **git reset --hard**: Deshace todos los cambios en el directorio de trabajo.
    - **git revert <commit>**: Crea un nuevo commit que deshace los cambios de un commit anterior.
- **Etiquetas**
    - **git tag <nombre-etiqueta>**: Crea una nueva etiqueta.
    - **git tag -d <nombre-etiqueta>**: Elimina una etiqueta.